연습문제 정답과 풀이

합성곱 신경망 (CNN) · 딥러닝개론및연습 2026년 2학기

강의노트 끝의 연습문제 7개의 정답과 풀이이다. 문제를 다시 적고 그 아래에 풀이를 쓴다. 2, 4, 5, 6, 7번의 결과는 `연습문제_정답/code/cnn_exercise_runs.py`를 이 컴퓨터에서 실제로 실행하여 확인하였다 (손으로 계산한 값과 같다).

1. 다음 합성곱 계층의 출력 크기를 구하라. (a) 입력 $(32, 32)$, 필터 $(5, 5)$, 패딩 2, 스트라이드 1. (b) 입력 $(227, 227)$, 필터 $(11, 11)$, 패딩 0, 스트라이드 4 (AlexNet의 첫 번째 층). (c) 입력 $(28, 28)$에 2×2 최대 풀링 (스트라이드 2)을 두 번 적용한 결과.

풀이. $OH = \left\lfloor \frac{H+2P-FH}{S} \right\rfloor + 1$을 쓴다 (가로도 같다).

- (a) $\frac{32+4-5}{1} + 1 = 32$, 출력 $(\mathbf{32}, \mathbf{32})$. 필터 5×5에 패딩 2를 주면 크기가 유지된다.
- (b) $\frac{227+0-11}{4} + 1 = \frac{216}{4} + 1 = 55$, 출력 $(\mathbf{55}, \mathbf{55})$. (224로 계산하면 $\frac{213}{4}$가 정수가 아니다.)
- (c) $28 \to \frac{28-2}{2} + 1 = 14 \to \frac{14-2}{2} + 1 = 7$, 출력 $(\mathbf{7}, \mathbf{7})$.

2. 입력 $\begin{pmatrix} 1 & 0 & 2 & 1 \\ 0 & 3 & 1 & 0 \\ 2 & 1 & 0 & 4 \\ 1 & 0 & 1 & 2 \end{pmatrix}$에 필터 $\begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$를 스트라이드 1, 패딩 0으로 적용한 결과를 손으로 계산하라. 같은 입력에 2×2 최대 풀링 (스트라이드 2, 패딩 0)을 적용한 결과도 구하라.

풀이. 출력 크기는 $4 - 2 + 1 = 3$이므로 3×3이다. 필터를 겹친 2×2 영역의 왼쪽 위 값에서 오른쪽 아래 값을 빼면 된다: $y_{ij} = x_{ij} - x_{i+1,j+1}$.

$$\begin{pmatrix} 1-3 & 0-1 & 2-0 \\ 0-1 & 3-0 & 1-4 \\ 2-0 & 1-1 & 0-2 \end{pmatrix} = \boxed{\begin{pmatrix} -2 & -1 & 2 \\ -1 & 3 & -3 \\ 2 & 0 & -2 \end{pmatrix}}.$$

최대 풀링은 겹치지 않는 2×2 영역 네 개의 최댓값이다.

$$\begin{pmatrix} \max(1,0,0,3) & \max(2,1,1,0) \\ \max(2,1,1,0) & \max(0,4,1,2) \end{pmatrix} = \boxed{\begin{pmatrix} 3 & 2 \\ 2 & 4 \end{pmatrix}}.$$

3. RGB 이미지 ($C = 3$)에 3×3 필터 64개를 쓰는 합성곱 계층의 매개변수(가중치와 편향) 수를 구하라. 입력이 $(3, 32, 32)$이고 패딩 1일 때, 같은 크기의 출력 $(64, 32, 32)$을 완전연결 계층으로 만든다면 가중치는 몇 개가 필요한가?

풀이. 필터 하나는 $(3, 3, 3)$으로 가중치 27개, 필터가 64개이고 편향은 필터마다 하나이므로

$$64 \cdot 3 \cdot 3 \cdot 3 + 64 = 1728 + 64 = \mathbf{1792}.$$

완전연결 계층이면 입력 $3 \cdot 32 \cdot 32 = 3072$개와 출력 $64 \cdot 32 \cdot 32 = 65536$개를 모두 잇는다.

$$3072 \times 65536 = \mathbf{201326592}\ (\text{약 } 2\text{억})$$

개의 가중치가 필요하다. 합성곱 계층의 약 11만 배이다. 합성곱 계층은 (1) 출력 하나가 가까운 3×3 영역만 보고 (국소 연결), (2) 모든 위치가 같은 필터를 쓰기 (가중치 공유) 때문에 매개변수가 이렇게 적다.

4. 2번 문제의 최대 풀링에서 출력 쪽 gradient가 $\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$ 일 때, 입력 쪽 gradient (4×4)를 구하라.

풀이. 최대 풀링의 역전파는 각 영역에서 **최댓값이었던 위치에만** gradient를 그대로 보내고 나머지는 0이다 (최댓값이 아닌 원소는 조금 바뀌어도 출력이 변하지 않기 때문이다). 최댓값의 위치는 왼쪽 위 영역의 3 (1행 1열), 오른쪽 위의 2 (0행 2열), 왼쪽 아래의 2 (2행 0열), 오른쪽 아래의 4 (2행 3열)이다 (0부터 센다). 따라서

$$\frac{\partial L}{\partial X} = \boxed{\begin{pmatrix} 0 & 0 & 2 & 0 \\ 0 & 1 & 0 & 0 \\ 3 & 0 & 0 & 4 \\ 0 & 0 & 0 & 0 \end{pmatrix}}.$$

ReLU의 역전파와 비슷하게, 순전파에서 "통과한" 곳으로만 gradient가 흐른다.

5. 길이 5인 1차원 입력 $(x_1, \ldots, x_5)$에 길이 3인 필터를 스트라이드 1로 적용하면 출력은 3개이다. im2col 행렬 (3×3)을 쓰고, x_3이 몇 번 등장하는지 세어 보라. 이로부터 col2im에서 gradient를 **더해야** 하는 이유를 설명하라.

풀이. 출력 하나가 보는 입력 구간을 한 행으로 펼친다.

$$\text{col} = \begin{pmatrix} x_1 & x_2 & x_3 \\ x_2 & x_3 & x_4 \\ x_3 & x_4 & x_5 \end{pmatrix}, \qquad \begin{pmatrix} y_1 \\ y_2 \\ y_3 \end{pmatrix} = \text{col} \begin{pmatrix} w_1 \\ w_2 \\ w_3 \end{pmatrix}.$$

x_3은 **3번** (세 행 모두) 등장한다. x_2, x_4는 2번, x_1, x_5는 1번이다.
역전파에서 행렬곱의 gradient $\frac{\partial L}{\partial \text{col}}$는 col의 각 **칸**에 대한 gradient이다. 그런데 세 칸이 모두 같은 변수 x_3의 복사본이다. x_3은 y_1, y_2, y_3 세 출력에 모두 영향을 주므로, 다변수 연쇄법칙에 따라

$$\frac{\partial L}{\partial x_3} = \frac{\partial L}{\partial y_1} w_3 + \frac{\partial L}{\partial y_2} w_2 + \frac{\partial L}{\partial y_3} w_1$$

처럼 세 경로의 gradient를 **더해야** 한다. col2im은 각 칸의 gradient를 원래 위치로 되돌리면서 같은 위치로 가는 값들을 더한다. (덮어쓰면 마지막 경로의 gradient만 남아 틀린다.)
순전파에서 한 변수가 여러 곳으로 복사되면 역전파에서는 그 gradient들을 더한다는 원리이다.

6. SimpleConvNet의 합성곱 계층에 패딩 2를 주도록 고쳐 보라. `conv_out`, `pool_out`, `flat`은 각각 얼마가 되는가? Convolution 계층에 `pad`를 넘기는 부분과 `flat`의 계산을 고친 뒤, `conv_basics.py`의 gradient 확인으로 검증하라.

풀이. MNIST (28×28, 필터 5×5 30개)에서

$$\texttt{conv_out} = 28+2\cdot 2-5+1 = \mathbf{28}, \quad \texttt{pool_out} = 28 \,//\, 2 = \mathbf{14}, \quad \texttt{flat} = 30\cdot 14\cdot 14 = \mathbf{5880}.$$

패딩이 없을 때는 24, 12, 4320이었다. `W2`가 4320×100에서 5880×100으로 커져 매개변수가 $433890 \to$ **589890**개가 된다.

고칠 곳 (`conv_layers.py`의 `SimpleConvNet.__init__`).

```
def __init__(self, input_shape=(1, 28, 28), filter_num=30, filter_size=5,
             pad=2, hidden_size=100, output_size=10, weight_init_std="he"):
    C, H, W = input_shape
    conv_out = H + 2 * pad - filter_size + 1        # 28 + 4 - 5 + 1 = 28
    pool_out = conv_out // 2                        # 14
    flat = filter_num * pool_out * pool_out         # 30 * 14 * 14 = 5880
    ...
    self.layers = [
        Convolution(self.params["W1"], self.params["B1"], pad=pad),
        ...
```

`Convolution`에 `pad`를 넘기지 않으면 순전파의 출력이 24×24로 남아, `Flatten`의 출력 4320이 `W2`의 행 수 5880과 맞지 않아 오류가 난다. 반대로 `flat`을 고치지 않으면 `W2`가 4320행이어서 같은 오류가 난다. 두 곳을 함께 고쳐야 한다.

gradient 확인. `conv_basics.py`와 같은 작은 network (입력 $(2, 8, 8)$, 3×3 필터 3개, 은닉 5, 출력 4, 데이터 5개, 같은 씨앗)에 패딩 2를 주었다. 이때 $\texttt{conv_out} = 8+4-3+1 = 10$, $\texttt{pool_out} = 5$, $\texttt{flat} = 75$이다.

매개변수	shape	$\max \lvert$역전파 $-$ 수치 미분$\rvert$
`W1`	$(3, 2, 3, 3)$	3.3×10^{-10}
`B1`	$(3,)$	5.0×10^{-10}
`W2`	$(75, 5)$	5.1×10^{-10}
`B2`	$(1, 5)$	2.4×10^{-10}
`W3`	$(5, 4)$	2.7×10^{-10}
`B3`	$(1, 4)$	1.5×10^{-10}

모든 차이가 10^{-9}보다 작으므로 패딩을 넣은 역전파도 정확하다. 패딩한 칸에 대한 gradient는 `col2im`이 패딩을 잘라 내며 버린다. 패딩은 상수 0이라 학습할 값이 아니기 때문이다.

7. `mnist_cnn_torch.py`를 고쳐 LeNet과 비슷한 network (Conv 5×5 6개, 패딩 2 $\to$ ReLU $\to$ MaxPool $\to$ Conv 5×5 16개 $\to$ ReLU $\to$ MaxPool $\to$ FC 120 $\to$ ReLU $\to$ FC 84 $\to$ ReLU $\to$ FC 10)를 만들고 학습시켜라. 각 계층을 지난 뒤의 shape을 표로 정리하고, 매개변수 수와 검증 정확도를 SimpleConvNet과 비교하라.

풀이. `mnist_cnn_torch.py`의 `model`만 다음으로 바꾼다 (나머지: Adam $\alpha = 0.001$, 미니배치 100, 10 epoch, 같은 데이터 분할).

```
model = nn.Sequential(
    nn.Conv2d(1, 6, kernel_size=5, padding=2), nn.ReLU(), nn.MaxPool2d(2),
    nn.Conv2d(6, 16, kernel_size=5), nn.ReLU(), nn.MaxPool2d(2),
```

```
    nn.Flatten(),
    nn.Linear(16 * 5 * 5, 120), nn.ReLU(),
    nn.Linear(120, 84), nn.ReLU(),
    nn.Linear(84, 10),
).to(device)
```

계층별 shape과 매개변수 (ReLU는 shape을 바꾸지 않으므로 생략).

계층	출력 shape	매개변수 수
입력	$(1, 28, 28)$	
Conv 5×5 6개, 패딩 2	$(6, 28, 28)$	$6 \cdot 1 \cdot 25 + 6 = 156$
MaxPool 2×2	$(6, 14, 14)$	
Conv 5×5 16개	$(16, 10, 10)$	$16 \cdot 6 \cdot 25 + 16 = 2416$
MaxPool 2×2	$(16, 5, 5)$	
Flatten	400	
FC	120	$400 \cdot 120 + 120 = 48120$
FC	84	$120 \cdot 84 + 84 = 10164$
FC	10	$84 \cdot 10 + 10 = 850$
합계		**61706**

SimpleConvNet과 비교 (GPU에서 같은 스크립트로 두 network를 차례로 학습, 미니배치 순서의 씨앗 동일).

network	매개변수 수	검증 정확도	test 정확도	시간
SimpleConvNet	433890	**0.9890**	0.9889	21초
LeNet 비슷한 network	61706	0.9872	0.9897	26초

해석.

- LeNet 비슷한 network는 매개변수가 SimpleConvNet의 약 $\frac{1}{7}$ (14%)인데 정확도는 거의 같다. 검증 정확도는 0.18%p (이미지 9장) 낮고 test 정확도는 0.08%p 높아서, 어느 쪽이 낫다고 할 수 없는 차이이다.
- 매개변수가 적은 이유는 풀링을 두 번 하여 Flatten의 출력이 400으로 작기 때문이다. SimpleConvNet은 4320×100인 첫 FC 계층에 매개변수의 99.6%가 있다. 합성곱 층을 하나 더 쌓아 특징 맵을 줄이면, 같은 정확도를 훨씬 적은 매개변수로 얻는다 (다음 주 DeepConvNet의 생각과 같다).
- 학습 시간은 LeNet 쪽이 조금 더 길다. 매개변수는 적지만 합성곱 층이 둘이고, 계층 수가 많아 GPU에서 작은 연산을 여러 번 하기 때문이다. 매개변수의 수와 계산 시간은 다르다.
- 씨앗 하나로 한 번씩 학습한 결과이고, GPU 계산은 실행마다 조금 다를 수 있다 (이 컴퓨터에서 SimpleConvNet을 다른 때 실행하면 test 정확도가 0.9861, 0.9886이었다).